

继承与派生

基类与派生类

若B是A，则可由A为基类派生出B

- 派生类是通过对基类进行修改和扩充得到的
- 派生类定义后不依赖于基类，可以独立使用
- 派生类拥有基类的所有成员函数和成员变量
- 派生类可以扩充新的成员函数和成员变量

```
class 派生类名: public 基类名{

};
```

派生类的内存空间

派生类对象的体积=基类对象的体积+自己的成员变量的体积

基类对象的存储位置位于派生类新增的成员变量之前

继承与复合

B是A，继承关系->派生类

B是A的特性，复合关系->封闭类

覆盖

定义：派生类可以定义一个和基类成员同名的成员

访问顺序：

在派生类中访问同名成员时，缺省的情况是访问派生类中定义的成员，要访问基类同名成员需要使用 作用域符号::

```
#include <iostream>
using namespace std;

// 基类
class Base {
public:
    void show() {
        cout << "这是基类中的 show()" << endl;
    }
}
```

```
};

// 派生类
class Derived : public Base {
public:
    void show() {
        cout << "这是派生类中的 show()" << endl;
    }
};

int main() {
    Derived d;

    d.show();           // 默认访问派生类中的 show()
    d.Base::show();     // 访问基类中的 show()

    return 0;
}
```

只要同名就会覆盖，即使参数表不同

当派生类中定义了一个与基类同名但参数表不同的成员函数时，基类中所有同名函数都会被隐藏，而不是与派生类函数共同形成重载。

此时，若要访问基类中的同名函数，必须使用作用域运算符 `::` 显式指出。

保护成员 protected

可被下列函数访问：

- 基类的成员函数
- 基类的友元函数
- 派生类的成员函数

派生类的构造函数

先构造基类，通过调用基类的构造函数（在初始化列表中）

```
#include <iostream>
using namespace std;

// 基类
class Person {
public:
```

```

    Person(string name) {
        cout << "基类构造函数，姓名: " << name << endl;
    }
};

// 派生类
class Student : public Person {
public:
    Student(string name, int age) : Person(name) {
        cout << "派生类构造函数，年龄: " << age << endl;
    }
};

int main() {
    Student s("张三", 20);
    return 0;
}

```

封闭派生类对象的构造函数

```

#include <iostream>
#include <string>
using namespace std;

// 普通类：用作成员对象
class Phone {
private:
    string number;

public:
    Phone(string num) : number(num) {
        cout << "Phone 构造函数被调用" << endl;
    }

    void showPhone() {
        cout << "电话: " << number << endl;
    }
};

// 基类
class Person {
private:
    string name;

```

```

    int age;

public:
    Person(string n, int a) : name(n), age(a) {
        cout << "Person 构造函数被调用" << endl;
    }

    void showPerson() {
        cout << "姓名: " << name << ", 年龄: " << age << endl;
    }
};

```

// 封闭派生类: 继承 Person, 包含 Phone 成员对象

```

class Student : public Person {
private:
    int studentId;
    Phone phone; // 成员对象

public:
    // 封闭派生类构造函数
    Student(string n, int a, int id, string phoneNum)
        : Person(n, a), // 1. 初始化基类
          phone(phoneNum), // 2. 初始化成员对象
          studentId(id) // 3. 初始化自身成员
    {
        cout << "Student 构造函数被调用" << endl;
    }

    void showStudent() {
        showPerson();
        phone.showPhone();
        cout << "学号: " << studentId << endl;
    }
};

```

```

int main() {
    cout << "==== 创建对象 =====> << endl;
    Student s("张三", 20, 10001, "138000001111");

    cout << "\n==== 显示信息 =====> << endl;
    s.showStudent();
}

```

```
    return 0;
}
```

构造顺序为：

1. 先构造基类部分
 2. 再构造成员对象
 3. 最后构造派生类自身
- 析构函数的执行顺序与构造函数相反

继承方式

public 继承

赋值兼容规则（针对public继承）

1. 派生类的对象可以赋值给**基类对象**
2. 派生类对象可以初始化**基类引用**
3. 派生类对象的地址可以赋值给**基类指针**

基类与派生类的指针

```
#include <iostream>
using namespace std;

// 基类
class Animal {
public:
    string name = "动物";
    void eat() {
        cout << "Animal::eat()" << endl;
    }
};

// 派生类
class Dog : public Animal {
public:
    string breed = "金毛";
    void eat() { // 覆盖基类函数
        cout << "Dog::eat()" << endl;
    }

    void bark() { // 派生类新增函数
```

```

        cout << "Dog::bark() 汪汪汪! " << endl;
    }
};

int main() {
    Dog dog;

    // ===== 派生类指针指向派生类对象 =====
    Dog* pDog = &dog;
    pDog->eat();    // Dog::eat()
    pDog->bark();   // Dog::bark()

    // ===== 基类指针指向派生类对象 =====
    Animal* pAnimal = &dog; // ✅ 允许
    pAnimal->eat();          // Animal::eat() (调用基类版本)
    // pAnimal->bark();       // ❌ 错误! 基类指针看不到派生类新增成员

    // ===== 派生类指针指向基类对象 =====
    Animal animal;
    // Dog* pDog2 = &animal; // ❌ 错误! 不允许

    return 0;
}

```

操作	结果
p->name	✅ 可访问 (基类成员)
p->eat()	✅ 可调用, 但调用的是 基类版本
p->breed	❌ 不可访问 (派生类新增成员)
p->bark()	❌ 不可调用 (派生类新增函数)

基类指针只能"看到"基类中定义的成员，**看不到派生类新增的成员。**

protected 继承

基类的**public成员**和**protected成员**成为派生类的**protected成员**

private继承

基类的**public成员**成为派生类的**private成员**，基类的**protected成员**成为派生类的**不可访问成员**

直接基类与间接基类

在继承关系中，一个类继承时所**直接写在类名后面**的基类，称为**直接基类**；如果某个基类不是当前类直接继承得到的，而是通过上一层继承关系间接获得的，则称为**间接基类**。

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "A 是基类" << endl;
    }
};

class B : public A {
public:
    void showB() {
        cout << "B 直接继承 A" << endl;
    }
};

class C : public B {
public:
    void showC() {
        cout << "C 直接继承 B" << endl;
    }
};

int main() {
    C c;

    c.showA(); // 来自 A
    c.showB(); // 来自 B
    c.showC(); // 自己的成员

    return 0;
}
```

成员包括：

1. 派生类自己定义的成员
2. 直接基类中的所有成员

3. 所有间接基类的所有的成员